01 / ROLE AND ROADMAP

Samarth's Phase 1 work guide

PROJECT 11 / AUTOMATIC CLOTHING CATEGORY RECOGNITION

Inventory Image Tagging via Handcrafted Features and a Small CNN
Phase 1 focus: classical machine learning on Fashion-MNIST.

Your role: SVM, Random Forest and experiment lead

Run a tiny end-to-end experiment before starting costly training.

The five-person division

Member	Primary ownership
Nakul	Integration, literature and report Repository, master notebook, report assembly
Sudeshhna	Data quality and EDA Loader, fixed split, audit, EDA figures
Puneet	Features, Logistic Regression and kNN Pixel/HOG features and four model rows
Samarth	Random Forest, SVM and run harness Four model rows and experiment logs
Neelanshu	Evaluation and error analysis Metrics, timing, complexity, error figures

How to use this guide

Read pages 2-4 first, then work through your practical pages in order. Pages 13-16 explain handoffs, viva preparation, the proposed schedule and sources. Your guide is a work plan and learning manual, not a completed experiment or a submission-ready report.

All five members must understand the entire pipeline. Ownership means you implement and explain an area; it does not mean the others can skip learning it. Each member also writes their report section, prepares their slides, and reviews another member's work.

Prepared 27 October 2026. All workload estimates, code organization, hyperparameter grids and internal milestones are recommendations, not faculty rules. No model scores have been invented.

02 / SHARED FOUNDATIONS

What Phase 1 actually requires

The supplied CSAI instructions are the course requirements. Your project description supplies the technical problem. This guide adds a practical way to satisfy both; it does not turn its suggestions into extra faculty requirements.

Course requirement	Evidence the team should prepare
Literature review	A cited comparison of relevant sources and the gap your experiments investigate.
Dataset quality and EDA	Shape/range/label checks, class counts, sample images and a written audit.
Feature engineering	Normalized flattened pixels and a justified HOG configuration.
Theoretical rigor	Explain each model, its assumptions, tuning choices and limitations.
Model application	Two representations with Logistic Regression, kNN, Random Forest and SVM.
GitHub and code quality	Reproducible code, clear README, saved settings, outputs and contribution evidence.
Presentation and individual viva	Every member presents their work and explains the shared experiment.
Project report	A coherent report in LaTeX/IEEE format, using the course template if supplied.

Scope boundary

Complete the classical ML component now. Save the CNN, pooling, dropout, CNN early stopping, AML-versus-CNN experiments and any implemented hybrid approach for Phase 2. A brief future-work paragraph is enough at this stage. The Phase 1 comparison is pixels versus HOG across four classical classifiers.

The attachment gives no Phase 1 evaluation date, page limit, presentation duration, grading weights, minimum accuracy, or required number of papers. Its September dates concern project/team selection. Check the course channel for later evaluation announcements; do not treat the proposed schedule here as an official deadline.

03 / SHARED FOUNDATIONS

Machine learning from zero

Here, **AML** means the classical/applied machine-learning part described in your project. You do not hand-write a rule saying "sleeves mean shirt." Instead, you show a model labeled examples so it can learn a rule from image measurements.

Term	Plain meaning in this project
Sample; label	One clothing image; its correct category number. The label is the answer.
Feature; representation	A numeric input. One image becomes either 784 pixel values or a HOG vector.
X and y	X is the feature matrix: one row per image. y is the matching list of labels.
Model; fit; predict	A learned decision rule; learn from X and y; assign labels to new X.
Parameter	Something learned during fitting, such as a Logistic Regression coefficient.
Hyperparameter	A setting chosen before fitting, such as k in kNN or C in an SVM.
Training / validation / test	Practice examples / development exam / final independent exam.
Overfitting	Doing very well on training data but failing to generalize to held-out data.
EDA	Exploratory data analysis: inspect the data before deciding how to model it.
Reproducibility	Another person can repeat the process using the same data, code and settings.

Walk through a single image

A grayscale image is a small grid of brightness values. Divide its raw pixel values by 255 to put them on a 0-1 scale. Flatten the grid to a vector, or compute a HOG description of local edge directions. A fitted classifier then maps that vector to one of the category IDs. Map the ID to the correct name for display.

Your first learning exercise

Before your assigned work, use a notebook to print an array's shape, show one image, print its label, and call fit then predict on a tiny example. Explain aloud why a row of X must stay attached to the same row of y. If that connection is broken, model training becomes meaningless.

Important nuance: flattening does not erase pixel values or their recoverable ordering. It gives a conventional classifier no built-in local-neighborhood or translation structure. HOG encodes local geometry, but its feature rule is fixed before classifier training.

04 / SHARED FOUNDATIONS

One shared, fair experiment

Freeze this protocol before comparing models

- Keep the official training and test partitions separate. Create a stratified 80/20 split of the official training partition with random_state=42: 48,000 fit images and 12,000 validation images. Save the indices once; everyone reuses them.
- Use two fixed representations and four model families: pixels/HOG × Logistic Regression/kNN/Random Forest/SVM. This produces eight main comparison rows. Add one DummyClassifier reference row outside those eight.
- Train on the fit partition, choose settings using validation macro F1, and use validation accuracy as a tie-breaker. If still tied, choose the simpler or faster option using validation measurements. Log the rule before seeing test scores.
- Fit learned scalers only on fit data. A fixed divide-by-255 transform and fixed per-image HOG extraction do not learn dataset statistics. Use a Pipeline for StandardScaler plus the classifier.
- After freezing one configuration per row and naming the validation-selected overall model, evaluate the eight frozen models on the official test partition. Do not choose new hyperparameters or a new overall winner from test results.

Training size and compute policy

Default: all eight final models use the same 48,000-image fit partition and the same 12,000-image validation partition. Keep those fitted models for final testing; do not silently refit some models on all 60,000 training images. This preserves the same split for Phase 2.

If compute makes this infeasible, agree before final runs on one saved stratified 12,000-image subset drawn only from the 48,000 fit images. Use that identical subset for all eight rows, retain the same validation/test partitions, and label the table "12,000 fit images per model." Exploratory subset runs must never masquerade as full-size results.

Required result package

Report accuracy, macro F1, recall for all ten classes, training time, inference time with a stated protocol, and a meaningful model-complexity description. Explain when a classical model has no neural-network-style parameter count. Save all eight rows, including weak runs; discuss shirt/T-shirt/pullover confusion using observed examples.

Protocol choices on this page are recommendations for consistency. The official split and the stated project metrics come from your project description; the specific validation fraction, seed and selection rule are proposed here.

05 / YOUR WORK

Your ownership and first session

You implement two model families that make very different decisions, and you make experiments traceable. Your four main rows are pixels+Random Forest, HOG+Random Forest, pixels+SVM and HOG+SVM. You also supply the common run harness used with Puneet's models.

What you personally build

- **src/models.py:** RF and SVM factories with explicit settings, coordinated with Puneet's LR/kNN factories.
- **src/experiments.py:** a runner that fits, predicts, times, logs warnings, saves artifacts and exports ID-aligned validation predictions.
- **notebooks/03_rf_svm.ipynb:** theoretical explanations, smoke runs, compact validation searches and your four frozen model rows.
- **Run manifest and report text:** executed settings, training sizes, completed/failed run status, theory, compute constraints and measured trade-offs. Review Nakul's reproduction instructions.

First 90 minutes

0-20 min: learn how fit and predict consume X and y. **20-40 min:** draw a tiny decision tree and explain its questions. **40-60 min:** draw two classes separated by a margin and explain what an SVM tries to do. **60-90 min:** train one RF and one SVM on small synthetic data, then print prediction shape and elapsed time.

Know what you depend on

Sudeshhna supplies raw data and saved fit/validation IDs. Puneet supplies the two feature functions and HOG settings. Neelanshu supplies the metric and prediction-file contract. If these are incomplete, build your runner on toy arrays first rather than invent a private split or feature definition.

Your handoff is accepted when

Neelanshu can trace every prediction to an original image ID and reproduce the row's metrics. Nakul can run your harness for one small model from the README. A failed run is logged honestly and cannot be confused with a successful frozen artifact.

Suggested effort: 20-26 hours plus unattended training. RBF SVM compute can dominate; decide the budget early instead of discovering it on the final day.

06 / YOUR WORK

Random Forest: theory you can explain

A decision tree asks a sequence of feature-threshold questions, such as whether a particular HOG bin is above a threshold. Leaves contain class information. A deep tree can fit very specific patterns and overfit. A Random Forest combines many randomized trees to reduce dependence on one unstable tree. [R8]

Understand the randomization

With bootstrap=True, each tree is fitted on a sampled training set drawn with replacement. At a split, max_features restricts the candidate features considered. Trees therefore differ. scikit-learn combines tree class-probability estimates and predicts from the average; a majority-vote explanation is useful intuition but not the exact implementation.

Understand splitting and regularization

For Gini impurity, Gini = 1 - sum_c p_c^2, where p_c is the fraction of a node's samples in category c. Splits seek cleaner child nodes. max_depth and min_samples_leaf constrain tree complexity. n_estimators changes how many trees are averaged; adding trees is different from letting each tree grow deeper.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

def make_rf(max_depth=None, min_samples_leaf=1):
    return Pipeline([
        ("scale", "passthrough"),
        ("model", RandomForestClassifier(
            n_estimators=200, max_depth=max_depth,
            min_samples_leaf=min_samples_leaf,
            max_features="sqrt", bootstrap=True,
            criterion="gini", random_state=42, n_jobs=1))])

rf = make_rf()
rf.fit(Xtr, ytr)
pred = rf.predict(Xva)
```

Why no learned scaler?

A tree splits features by thresholds, so conventional per-feature standardization is generally unnecessary for this baseline. Still use the shared normalized pixels or HOG features. Do not train RF directly on an image tensor if the interface expects a two-dimensional feature matrix.

A feature-importance score is not a causal explanation of clothing categories. Correlated features and the chosen importance method can distort its interpretation. For Phase 1, clear confusion examples are often more useful than a visually attractive but poorly justified importance plot.

07 / YOUR WORK

Tune and inspect Random Forest

A manageable validation plan

Start with 50 trees only for a smoke run. For the proposed final baseline use 200 trees, max_features="sqrt" and seed 42. If compute permits, test max_depth in {20, None} and min_samples_leaf in {1, 2}, four configurations per representation. Freeze the grid before test access. A smaller documented grid is acceptable if time is limited.

Setting	Effect to investigate
max_depth=20 vs None	Shallower trees may control overfitting; unrestricted trees can fit more complex boundaries and use more memory.
min_samples_leaf=1 vs 2	Larger leaves require a decision to be supported by more training samples; they can smooth noisy partitions.
200 trees	A reasonable starting ensemble size, not an accuracy guarantee or faculty requirement.
max_features="sqrt"	Randomized feature consideration encourages different trees; record it instead of relying on defaults.
n_jobs=1	Use a clear thread policy initially. If you use more threads, record it and keep final timing conditions consistent.

What to measure

Record validation accuracy, macro F1, all-class recall, fit time and artifact size. If computing a training score for overfitting analysis, state whether it covers the full fit set or a documented subset. Keep out-of-bag scores optional; they do not replace the shared validation set.

Explain the outcome rather than defend the method

If RF works better on pixels, a possible explanation is that individual intensity thresholds suit this data better than the chosen HOG aggregation. If HOG wins, fixed edge structure may help. These are plausible interpretations, not proven causes. Support them using observed confusion differences and careful qualification.

Record model structure

```
m = fitted_pipe.named_steps["model"]
tree_count = len(m.estimators_)
node_count = sum(t.tree_.node_count for t in m.estimators_)
depths = [t.tree_.max_depth for t in m.estimators_]
print(tree_count, node_count, min(depths), max(depths))
```

Tree count and total nodes describe structure. Serialized bytes describe storage. Do not call total nodes "the same parameter count" as LR weights. Give Neelanshu the counts and the fitted model so the report can label them correctly.

08 / YOUR WORK

SVM: margin, kernel and code

A Support Vector Machine seeks a decision boundary with a useful margin between classes while allowing some violations. Training examples that determine the boundary are called support vectors. An RBF kernel lets the decision depend on nonlinear similarity in the feature space. [R9]

Two hyperparameters to understand

C: controls how strongly violations are penalized; larger C can fit training data more tightly. **gamma:** controls the locality of the RBF similarity K(x,x') = exp(-gamma × ||x-x'||²). Very large gamma can make each example's influence narrow; very small gamma can make the decision too smooth. These effects interact with scaling and the data.

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

def make_svm(C=1.0, gamma="scale"):
    return Pipeline([
        ("scale", StandardScaler()),
        ("model", SVC(
            kernel="rbf", C=C, gamma=gamma,
            probability=False, cache_size=512,
            random_state=42))])

svm = make_svm()
svm.fit(Xtr, ytr)
pred = svm.predict(Xva)
```

Explain multi-class behavior

SVC trains pairwise one-versus-one classifiers internally for the ten categories. Do not claim it trains just one binary separator for all ten labels. The exact packed dual coefficient structure is implementation-specific; inspect the fitted object rather than assuming one simple ten-by-feature weight matrix.

Why probability=False?

Accuracy, macro F1 and class recall need predicted labels, not calibrated probabilities. Probability estimation adds work and is unnecessary for the requested baseline. If you display confidence later, a decision_function score is not automatically a probability. Label it honestly.

Your small tuning plan

Start with C=1 and gamma="scale". If compute permits, use C in {1, 10} and gamma in {"scale", 0.001}, four configurations per representation. gamma="scale" uses a feature-variance-based default from training input; it is not a universal best setting. Tune only on validation data.

Learned StandardScaler statistics must be fitted through the Pipeline on fit rows. Fitting the scaler before splitting creates leakage even if SVC.fit receives only training labels.

09 / YOUR WORK

Control SVM cost without invalidating results

RBF SVC can be expensive on tens of thousands of examples; its documented fit-time scaling can be worse than quadratic. Do not launch a huge grid or promise a completion time from a 2,000-image run. Estimate memory and runtime on progressively larger training subsets before committing. [R9]

Choose one explicit compute policy

Policy	What makes the comparison honest
Full profile	All eight final rows use 48,000 fit images. Keep searches compact and allow unattended runs. Record actual times.
Budget profile	All eight final rows use the identical saved 12,000-image fit subset. The main table says so. Validation and test partitions are unchanged.
Linear SVM alternative	If RBF is infeasible, agree before final testing to use LinearSVC for both SVM feature rows. Name them Linear SVM; this remains an SVM baseline but tests a different decision family from RBF.

Do not train SVC on fewer examples than the other models and bury the difference. Exploratory reduced-size SVM runs can be shown separately, with train_n prominently reported. The guide's preferred fallback is one shared budget profile, because it keeps the eight-row data comparison clean.

If the team chooses LinearSVC

```
from sklearn.svm import LinearSVC

linear_svm = Pipeline([
    ("scale", StandardScaler()),
    ("model", LinearSVC(C=1.0, max_iter=5000,
                        random_state=42))])
# Use this explicitly labeled alternative for both feature rows.
# Record solver/version defaults and any convergence warnings.
```

LinearSVC uses a different optimization implementation and a linear decision function. It does not share the RBF kernel's gamma setting or support-vector storage structure. Its multiclass default is one-versus-rest. If you adopt it, update the theory, report and complexity calculation together.

Avoid false shortcuts

- Do not impose a low iteration cap on SVC and treat an unconverged model as the final optimized result. Record convergence status and limitations.
- Avoid multiple heavy parallel fits on a memory-limited machine. cache_size is a kernel-cache budget in MB, not the full model's memory consumption.
- Let a completed benchmark, not a guess, determine whether the full or budget profile is practical. Agree and document the final policy before comparing test scores.

10 / YOUR WORK

Build the experiment harness

A run is one specific combination of data IDs, representation, preprocessing, model and settings. Give it a unique ID such as hog4_svm_C1_seed42_full. The harness makes those choices explicit and saves enough evidence to audit failures as well as successes.

```
import time, warnings
from sklearn.metrics import accuracy_score, f1_score

def fit_and_validate(pipe, Xtr, ytr, Xva, yva):
    with warnings.catch_warnings(record=True) as captured:
        warnings.simplefilter("always")
        start = time.perf_counter()
        pipe.fit(Xtr, ytr)
        fit_s = time.perf_counter() - start
        start = time.perf_counter()
        pred = pipe.predict(Xva)
        predict_s = time.perf_counter() - start
    return pred, {
        "fit_seconds": fit_s,
        "validation_predict_seconds": predict_s,
        "validation_accuracy": accuracy_score(yva, pred),
        "validation_macro_f1": f1_score(
            yva, pred, labels=np.arange(10),
            average="macro", zero_division=0),
        "warnings": [str(w.message) for w in captured],
    }
```

Xtr and Xva here are unscaled cached features, so fit_seconds includes the Pipeline's learned scaler and classifier fit but excludes fixed pixel/HOG extraction. Record feature extraction separately. This validation predict time is a coarse run log, not the final repeated inference benchmark Neelanshu will conduct.

What the surrounding runner must add

- Read and verify run configuration, source/split hashes, original IDs, train_n, feature dimension and software versions before calling this helper.
- Save a prediction CSV in the shared schema, a fitted model bundle, metadata and the run result. Write status="completed" only after all required artifacts are saved.
- Catch exceptions in the outer runner, record status="failed" and the exception, then stop or move to the next planned run. A partial metrics file must not be mistaken for success.
- Accept an explicit stage argument so smoke, validation and final test outputs cannot overwrite each other. Final test execution uses frozen artifacts and no fitting.

Coordinate file edits with Puneet. One person can own src/models.py while the other supplies reviewed functions, or use separate submodules and a shared import layer.

11 / YOUR WORK

Select configurations and freeze evidence

Maintain a development table

For every attempted configuration, record run_id, feature/config, estimator/settings, train_n, split hash, seed, validation accuracy, validation macro F1, fit time, prediction time, warnings and status. Sort only completed comparable runs when selecting. Do not discard failed settings from the experiment history.

A simple freeze process

- 1. Confirm all four of your rows have at least one completed run on the agreed final training size. Ask Puneet to confirm the same for their four rows.
- 2. For each representation/model row, choose the candidate with best validation macro F1 using the predefined tie-breaker. If there was a separate reduced-size exploratory stage, keep it out of the final-size ranking.
- 3. Save the exact fitted artifact and configuration as the row finalist. Do not silently refit on a larger training pool after reporting validation numbers.
- 4. Compare the eight finalists using validation metrics to record the overall selected model ID. Preserve the validation table and selection decision.
- 5. Hand Neelanshu a freeze manifest. They run final evaluation without adjusting settings based on test outcomes.

Your freeze manifest

Include: repository commit; environment file; split hash; training profile; feature configs; one run ID and model artifact path per row; metric-selection rule; validation-selected overall model ID; model-file hashes; validation prediction paths; known warnings and limitations. A manifest is a small JSON/text record, not a new model.

Write useful model comparisons

Compare pixels versus HOG within RF and within SVM. Explain how feature structure and scaling interact with each classifier. Then compare RF versus SVM using actual recall, runtime and storage evidence. Do not infer that one model family is universally superior from one controlled benchmark.

Review another person's work

Review Nakul's README in a fresh session: install dependencies, load data/splits and run one tiny experiment. Document any missing command, stale path or hidden prerequisite. Your own harness should make the reproduction path short enough for someone else to follow.

12 / YOUR WORK

Troubleshooting and your completion test

Symptom	Diagnosis and response
SVM is far slower than RF	Possible expected kernel cost. Inspect training size and runtime growth; reduce optional search before changing the comparison profile.
SVM performs unusually poorly	Check scaling, y alignment, HOG metadata, convergence and whether you accidentally reused a stale model.
RF trains well, validates poorly	Possible overfitting. Compare depth/leaf settings using validation; also check split and class counts.
Different metrics on same model	Check prediction IDs, partition, loaded artifact and version. Compare saved y_true/y_pred arrays directly.
Memory error during search	Too many parallel fits/copies or large caches. Run sequentially and select one explicit compute policy.
Result file exists but no model	Run may have failed mid-save. Use a completion flag and require all artifact paths before final selection.
Unconverged LinearSVC	Inspect the warning, scaling and iteration count. Increase the budget if justified, log the rerun, and state remaining limits.

Your final checklist

- Your four main rows share feature settings, image IDs and final training size with Puneet's four rows.
- You can explain RF randomization and Gini splitting, SVM margin/kernel behavior, C, gamma and the chosen multiclass implementation.
- Your harness records versions, settings, warnings, failures and timing boundaries, and exports correctly aligned predictions.
- The freeze manifest identifies eight artifacts and the validation-selected overall model. No private test-driven tuning has occurred.
- Your report text accurately names RBF SVC or LinearSVC and uses the correct complexity description for whichever was actually run.

Explain these concepts without notes

Why do many trees help? Why does a forest not require StandardScaler? What does gamma do? Why can kNN fit quickly but predict slowly while SVM has costly fitting? Why must reduced training size be visible in the results? Why is test-based model selection unfair?

Your concrete completion standard

Someone else can run your harness, reproduce your four rows, and inspect the exact configuration and warning history behind every frozen artifact.

13 / TEAM DELIVERY

Handoffs and shared file contract

Each person works on their own files, commits meaningful changes, and opens a review request or asks a teammate to review. Share generated evidence with file paths and the exact run ID; a screenshot in a group chat is insufficient.

Sender → receiver	What is handed over; acceptance check
Sudeshhna → everyone	Loader, labels, split_indices.npz, audit.md. Row counts and train/validation IDs agree.
Puneet → Samarth + Neelanshu	Feature functions, HOG settings, feature metadata. A sample ID yields the same vector everywhere.
Puneet + Samarth → Neelanshu	Fitted pipelines, validation predictions, run logs and feature metadata. Every prediction has a split and sample ID.
Neelanshu → Nakul + model owners	Verified tables, error figures, timing method and audit findings. Results trace back to saved predictions.
All four → Nakul	Their report sections, figure captions, references, contribution notes and slides.
Nakul → everyone	Integrated notebook, draft report and slides. Each member checks their section and practices the full demo.

Use these identifiers consistently

sample_id: original row number within the official partition, plus **source_split** = official_train or official_test. Train/validation IDs both refer to official_train. Never use the row number of a shuffled feature cache as the original sample ID.

Prediction CSV: run_id, stage, source_split, sample_id, y_true, y_pred. stage is val or test. Do not invent probabilities when a classifier only exposes decision scores. Preserve deterministic sample order.

Run record: run_id, owner, git_commit, seed, split_file_hash, train_n, val_n, feature_name, feature_dim, feature_config, scaler, estimator, hyperparameters, package_versions, fit_seconds, status, warnings and artifact_paths. Keep validation and test metrics in separate fields/files.

Results CSV: run_id, stage, train_n, feature, model, accuracy, macro_f1, fit_seconds, predict_ms_per_image, end_to_end_ms_per_image, serialized_bytes, complexity_type, complexity_value. Put per-class recall in a separate ten-row table per run. Use seconds, milliseconds and bytes explicitly.

Every handoff includes: what changed, how to run it, expected shapes, the evidence checked, and known limitations. The person receiving it must rerun a small example before accepting it.

14 / TEAM DELIVERY

Individual viva practice

Practice without reading the answers. Explain each idea simply, then open the relevant code or figure as evidence. These are suggested talking points, not answers to memorize word for word.

Everyone must answer these

What is the difference between validation and test?
Validation guides development. Test measures the frozen procedure on held-out examples. Looking at test failures and then tuning would contaminate that final assessment.

Why use both pixels and HOG?
They test whether a fixed description of local edges helps compared with raw intensities. HOG may help some classifiers and hurt others; the experiment must establish the result.

Why is accuracy alone insufficient?
It summarizes total correctness and can hide a weak category. Macro F1 averages class F1 values equally; per-class recall shows which categories are missed.

What can you honestly conclude about a CNN now?
Only that a CNN is a motivated Phase 2 direction. There is no evidence that your future CNN outperforms your ML baseline until that comparison is run.

Samarth: questions about your own work

How do RF and SVM differ?
RF averages randomized tree-based partitions; SVM constructs margin-based decisions, with RBF similarity for the kernel version. Explain the actual implementation used.

Why does SVM need scaling?
Distances and regularized decision rules depend on feature scales. The learned scaler is inside the Pipeline and is fitted only on fit rows.

How did you handle expensive training?
Describe the real runtime checks, compact search and chosen shared training profile. If LinearSVC replaced RBF SVC, name it explicitly and explain the theory change.

Five-minute live demonstration

Show your own file and explain one nontrivial choice. Run a small cell or load a saved artifact. Trace one image from its original ID through features to a prediction. Read a validation metric correctly. Explain one failure, one limitation and one improvement reserved for Phase 2. Everyone must also describe the eight-row experiment and the shared split.

15 / TEAM DELIVERY

Suggested 10-day work plan

This is a relative plan, not an official deadline. Budget roughly 18-26 focused hours per person plus unattended compute; beginners may need longer. If you have fewer days, reduce the optional search grid and parallelize member-owned work. Keep data integrity, all eight baseline rows, and honest reporting.

Day	Your focus	Team gate
1	Learn trees, margins and logs	All five understand scope and vocabulary
2	Set up model factories	Environment works; literature notes started
3	Build timing/run harness	Data/split/feature contracts agreed
4	Smoke-test RF and SVM	One tiny run reaches the evaluator
5	Tune four feature/model rows	Protocol and compute budget fixed
6	Run frozen-size comparisons	Eight comparable rows progressing
7	Export models and predictions	Configurations frozen; test run authorized internally
8	Write RF/SVM and tuning text	Verified figures and individual text ready
9	Review Nakul's reproduction steps	One full rehearsal; report compiles
10	Check warnings and run manifest	Fresh-run check; contribution log complete

Definition of done for every member

- Your files run in the agreed environment, your inputs and outputs are documented, and a teammate has reviewed one small run.
- Your report section explains what you did, why you did it, what evidence supports it, and what its limits are. Captions and citations are included.
- Your contribution log names real files, commits, experiments and reviews. It records actual work, not an artificial equal number of commits.
- You can explain your own implementation and the shared project during an individual viva. You have practiced a backup demo using saved outputs.

If a run fails, preserve the error and settings. Tell the team early and revise the compute plan consistently. Do not fabricate results, omit failed rows without explanation, or quietly change the split.

16 / TEAM DELIVERY

Sources and how to use them

Course source: CSAI_Project_Instructions_Phase_1_and_Phase_2.md, supplied by you. Its Phase 1 list and individual-viva rule govern the required work. Your pasted Project 11 description supplies the feature/model comparison and metrics. Neither source contains actual results for your team.

Technical references below were checked on 27 October 2026. Stable documentation can change; record the versions actually installed for your experiments. Reference IDs such as [R3] in this guide point to these links. Read and cite original material in your submitted report; this guide is not a substitute for a literature review.

[R1] Fashion-MNIST official repository: data files, label map and official partitions

[R2] Xiao, Rasul and Vollgraf (2017), Fashion-MNIST dataset paper

[R3] scikit-image: HOG API and settings

[R4] scikit-image: HOG worked explanation

[R5] scikit-learn: leakage and preprocessing pitfalls

[R6] scikit-learn: LogisticRegression

[R7] scikit-learn: KNeighborsClassifier

[R8] scikit-learn: RandomForestClassifier

[R9] scikit-learn: SVC

[R10] scikit-learn: model evaluation and metrics

[R11] scikit-learn: stratified train_test_split

[R12] Torchvision: FashionMNIST loader

[R13] Dalal and Triggs (2005): original HOG paper, CVPR

[R14] scikit-learn: LinearSVC alternative and multiclass behavior

[R15] LeCun et al. (1998): author-hosted document-recognition paper record

Suggested research reading beyond API pages

Read Dalal and Triggs (2005), "Histograms of Oriented Gradients for Human Detection" [R13], for the original descriptor. Its task is human detection, so do not transfer its scores to clothing classification. LeCun et al. (1998), "Gradient-Based Learning Applied to Document Recognition" [R15], is useful Phase 2 background; it does not require a Phase 1 CNN implementation.

Citation discipline

For each source record title, authors, year, task, representation, model, evaluation protocol and relevance. Separate a paper's results from your results. Repository leaderboard entries are not automatically controlled comparisons or verified targets. Use actual bibliographic metadata in your LaTeX .bib file; never cite an unread paper as evidence for a specific finding.

Code blocks are starter recipes to adapt and run in your project environment. They have not been run on Fashion-MNIST as part of creating these guides. Toy calculations are illustrative, not measured project performance.